# FirmEx: Firmware Verification and Runtime Anomaly Detection Framework for Embedded Devices

Gandham Sai Santhosh
Indian Institute of Technology Madras
Chennai, India
santhoshgandham256@gmail.com

Haresh Dagale
Indian Institute of Science
Bengaluru, India
haresh@iisc.ac.in

Keerthana Jayaprakash
Indian Institute of Science
Bengaluru, India
keerthanajp27@gmail.com

## ABSTRACT

Detecting anomalous sensor behavior and verifying firmware integrity in critical embedded devices is essential for safety and security, yet existing solutions lack these capabilities in a single, automated, cross-protocol framework. Current anomaly detection schemes often lack robustness to noise and drift, while firmware verification workflows remain manual and protocol-specific.

We present FirmEx, a modular hardware-based framework that integrates passive real-time sensor monitoring, multi-protocol firmware extraction with automated integrity verification. FirmEx's FPGA-assisted bus sniffer captures structured sensor data from respective bus interfaces without disrupting normal operation. A Raspberry Pi processing unit runs a robust adaptive Z-score anomaly detector and coordinates firmware acquisition via a custom FTDI-based board supporting JTAG, SWD, SPI, UART, and I$^2$C. Our GUI orchestrates both anomaly detection and firmware verification without command-line intervention.

FirmEx demonstrated 94.73% precision with a 0.004% false positive rate and a median detection latency of 362.24 ms across six sensor types for anomaly detection, and achieved a 95% firmware dump success rate with a 2.5× improvement in extraction speed over manual methods in firmware analysis across 19 devices. These results demonstrate FirmEx's ability for rapid, low-overhead security monitoring and integrity verification in multi-protocol supporting, heterogeneous application environments.

## CCS CONCEPTS

• **Hardware → Embedded systems**; **Hardware security implementation**; • **Security and privacy → Embedded systems security**; **Intrusion/anomaly detection**.

## KEYWORDS

Embedded security, Firmware verification, Firmware extraction, Anomaly detection, Passive monitoring, Statistical anomaly detection

**ACM Reference Format:**
Gandham Sai Santhosh, Haresh Dagale, and Keerthana Jayaprakash. 2026. FirmEx: Firmware Verification and Runtime Anomaly Detection Framework for Embedded Devices . In *Proceedings of ACM Asia Conference on Computer and Communications Security (ASIACCS 2026).* ACM, New York, NY, USA, 12 pages.


*ASIACCS 2026, 2026, India*


## 1 INTRODUCTION

Embedded systems are an integral part of IoT and they are increasingly used for critical domains like industrial control, healthcare and automotive, yet they lack robust security safeguards. Research has demonstrated that exposed debug interfaces such as UART, JTAG/SWD, SPI, and I$^2$C pose severe security risks, enabling firmware extraction and system compromise in deployed devices through timing-based bypasses, direct memory access, or injection of malicious updates [1–3]. Traditional software-based defenses, such as OS-level intrusion detection or antivirus programs, lack visibility into low-level operations and on-chip behavior, leaving them unable to detect firmware-level threats. These defenses may themselves be susceptible to tampering.

Furthermore, embedded and cyber-physical devices often incorporate sensors connected via using external embedded interfaces that can be spoofed or tampered with through hardware-based attacks such as clock-glitching or man-in-the-middle interception on I$^2$C [4], allowing attackers to feed false data into control systems.

Ensuring the trustworthiness of such devices requires detecting both malicious firmware modifications and sensor data manipulation in real time. However, existing solutions tend to address these concerns in isolation: firmware integrity verification is often performed manually or offline[5–7], while runtime anomaly detection can be susceptible to noise, environmental drift, or spoofing[8]. In practice, this leaves a significant security gap; the literature survey did not reveal any integrated, automated, and continuous approach to both firmware verification and sensor anomaly detection across diverse hardware platforms.

To address this gap, we present FirmEx, a unified embedded framework that provides two complementary protections. First, an FPGA-based sniffer passively intercepts bus communication between target embedded device and sensors to capture live sensor data. This is forwarded to a Raspberry Pi running an adaptive Z-score anomaly detection model capable of filtering/flagging transient noise data separately while identifying statistically significant deviations. Second, a custom FT2232H-based multi-protocol interface board, also referred to as the "FirmEx board," supports JTAG/SWD, SPI, UART, and I$^2$C for automated firmware extraction, with SHA-256 hash computation and comparison against trusted images handled by the Raspberry Pi. This automation removes the need for protocol-specific manual workflows, reducing operator error and increasing repeatability.

A Python/Tkinter-based graphical interface unifies both capabilities, enabling operators to initiate passive monitoring, trigger firmware extraction, verify image integrity, and view live

anomaly alerts from a single dashboard. By combining real-time, non-intrusive sensor monitoring with automated multi-protocol firmware verification, FirmEx addresses a critical gap in embedded security.

The rest of the paper is organized as follows. Section( 2) situates FirmEx within existing efforts by outlining prior approaches to firmware extraction, anomaly detection, and bus sniffing, highlighting the gaps our framework addresses. Section( 3) then provides a high-level overview of FirmEx, introducing its core components and how they interact. Building on this, Section( 4) delves into the design methodology, detailing how the monitoring, acquisition, and analysis modules are realized in practice. Section( 5) evaluates the robustness of FirmEx through simulated attacks and security analyses, while Section( 6) presents quantitative results demonstrating its precision, efficiency, and reliability across diverse devices. Finally, Section( 7) reflects on the broader implications of our findings and identifies promising avenues for extending this work.

### 1.1 Contributions

The main contributions of this paper include the following:

- We propose a modular non-invasive real-time monitoring system, integrating passive sensor sniffing with automated firmware acquisition and verification. 18 out of 19 devices tested showed successful firmware dump with ∼2.5 times faster workflow.
- We propose an adaptive Z-score detection that addresses noise, transient spikes, and slow baseline drift in live sensor streams, enabling low-overhead anomaly alerts on diverse sensors. Tested across 6 sensor types with 200 baseline (training) samples and 100 injected anomalies (evaluated on 1000 samples), FirmEx achieved 94.73% Precision (FPR 0.004%) with median detection latency of 364.24 ms.
- We devise a hardware interface based on the FT2232H that allows firmware extraction through JTAG, SWD, SPI, UART, and $I^2C$. The board supports switching between voltage levels and signal lines, allowing firmware dumps from heterogeneous devices.
- A GUI that manages both anomaly monitoring and firmware verification, eliminating the need for manual CLI operations and making FirmEx practical for field deployment.

## 2 BACKGROUND AND RELATED WORK

### 2.1 Firmware Extraction Methods

Firmware extraction is critical for verifying the integrity of embedded devices. Common interfaces such as JTAG, SWD, UART, SPI, and $I^2C$ expose internal memory or flash components to external access, but each comes with limitations[9].

JTAG offers comprehensive memory access and debugging capabilities but usually requires external adapters (e.g., SEGGER J-Link) and complex pin mapping [10]. SWD, a two-wire alternative used in ARM Cortex micro controllers, is more compact but depends on proper signal timing and vendor-specific tooling[11].

UART boot-loaders provide lightweight firmware access, especially in constrained devices, but are only available if the boot-loader remains unprotected[12]. SPI flash dumping is widely used for accessing external NOR flash in routers and IoT devices. However, it often requires SOIC clips, level shifters, and programmers like the CH341A [13]. While $I^2C$ is not typically used for firmware access, it can leak binary data when EEPROMs or sensors store executable code[14].

Tools such as OpenOCD, URJTAG, Flashrom, and PyFTDI support communication over these interfaces. However, they require careful voltage handling, manual setup, and interface-specific commands. This fragmented tooling increases the setup complexity and risk of human error—highlighting the need for a unified, multi-protocol firmware extraction platform that standardizes access across multiple embedded devices. Existing tools such as SEC Xtractor [15], Attify Badge [16], Bus Pirate [17], and Tigard [18] support firmware extraction over interfaces like SPI, UART, and JTAG. However, these tools generally lack unified automation and require users to switch manually between protocols and tools. These modular workflows increase setup time, introduce manual error, and hinder scalability for real-time or batch analysis.

### 2.2 Runtime Anomaly Detection in Embedded Systems

Machine learning models such as Isolation Forest, k-means clustering, and autoencoders have been used to detect anomalies in embedded sensor data [22]. However, these approaches often require offline training, high computational resources, or are optimized for high-dimensional datasets rather than real-time sensor streams[23, 24].

In our preliminary testing, the Isolation Forest model effectively detected major spikes but consistently missed subtle deviations (4) —precisely the kind of anomalies that often indicate spoofing or tampering in embedded systems.

To address this, we adopt a lightweight, statistical anomaly detection approach based on Z-score analysis. This model operates in real time, builds a dynamic baseline from incoming data, and is sensitive to fine-grained variations. It includes a spike-filtering mechanism to flag abrupt noise artifacts, such as those caused by UART de-synchronization, separately from anomalies.

*We discuss more about this here [A.1] in detail.*

### 2.3 Existing Sniffers and Limitations

Hardware protocol sniffers are widely used for monitoring communication buses such as $I^2C$, SPI, UART, and USB in debugging, reverse engineering, and security analysis. Tools like logic analyzers [20], Total Phase Beagle[19], and micro controller-based sniffers like the Bus Pirate[17] provide visibility into digital communications, but often suffer from limited buffer sizes, lack of real-time decoding, and poor support for high-speed or multi-master scenarios. Many commercial sniffers are also cost-prohibitive and rely on proprietary software, reducing flexibility and limiting their applicability in open research settings [21].

Wright [25] documents the use of tools such as the Bus Pirate and logic analyzers for firmware extraction from embedded systems. These tools require manual interpretation, lack automation, and struggle to intercept traffic reliably in real time. Van Ieperen [26] similarly shows that while the Bus Pirate can successfully sniff $I^2C$ communication between an Arduino and BMP280 sensor, it tends to drop bytes under load, making it less reliable than dedicated logic analyzers.

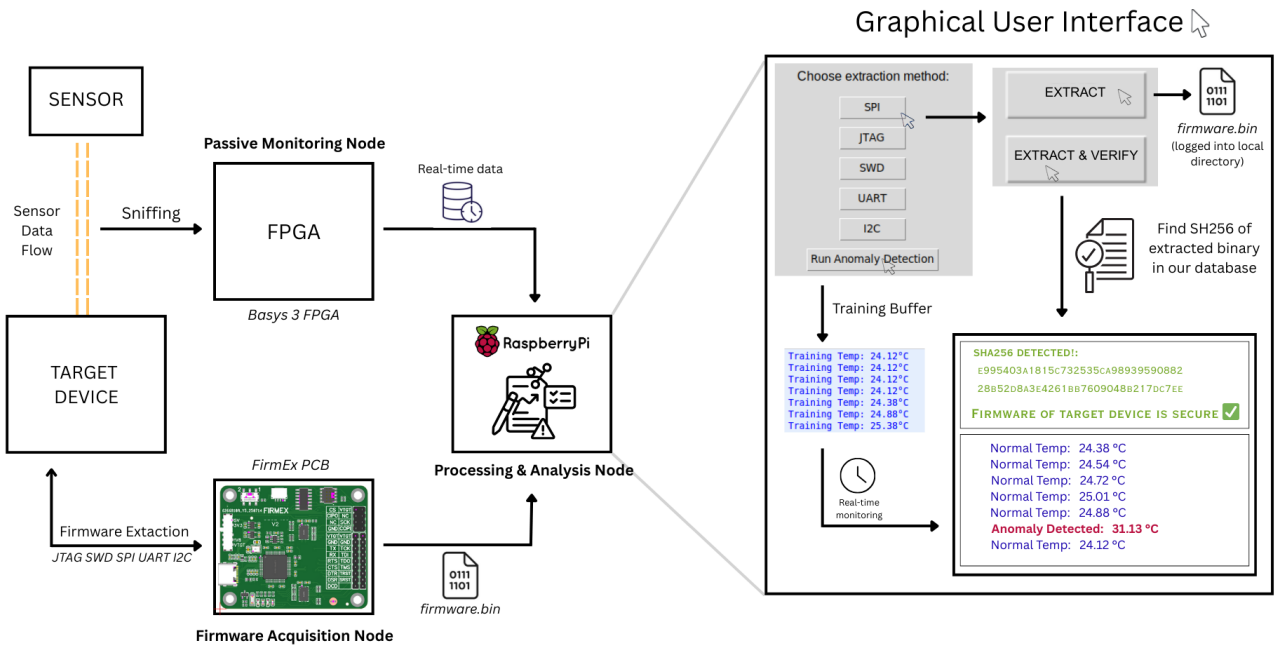


**Figure 1: The high-level embedded architecture of FirmEx summarizing the entire workflow of firmware integrity check and real-time anomaly detection all being carried out in a single, automated, non-intrusive, cross-protocol user interactive setup.**

Other studies explore non-invasive attacks on $I^2C$ buses, including hardware Trojans that leak or tamper with data without altering the physical device [27], and rogue devices that passively intercept communication between master and slave nodes [28]. These works confirm the feasibility of passive monitoring on $I^2C$ lines, but most are limited to a single protocol [29, 30].

Most existing tools provide limited automation and real-time capabilities, making it challenging to monitor and analyze bus activity efficiently. In this work, we present a lightweight, real-time monitoring solution implemented entirely in hardware.

## 3 SYSTEM OVERVIEW

The proposed framework is built around a modular, distributed architecture composed of three core nodes: a Passive Monitoring Node, a Firmware Acquisition Node, and a Processing and Analysis Node. These nodes operate collaboratively to extract and verify the integrity of firmware in embedded systems without interrupting their operation. This combination allows FirmEx to operate non-intrusively, detect malicious or faulty sensor behavior in real time, and concurrently validate firmware integrity across multiple hardware protocols. Refer to (1) for a visual overview.

### 3.1 Passive Monitoring Node

This node is designed to observe communication activity between different components of an embedded device, such as sensors and controllers. It listens non-intrusively to ongoing data exchanges and forwards structured observations for further analysis. By remaining entirely passive, it ensures that the operation of the target device is not altered or disrupted in any way.

### 3.2 Firmware Acquisition Node

This node provides controlled access to the internal storage or memory of the target system through industry-standard communication points. It acts as an adaptable interface layer capable of retrieving firmware data from a wide range of devices, regardless of the underlying hardware architecture. Its goal is to standardize and automate the acquisition process, reducing reliance on multiple external tools and manual intervention.

### 3.3 Processing and Analysis Node

This node acts as the central coordinator, receiving data from both the monitoring and acquisition nodes. It performs behavior analysis to detect irregularities and verifies firmware integrity by comparing extracted data against trusted references. The system isolates relevant portions of the acquired data for accurate comparison, minimizing false positives due to extraneous content. An interactive GUI enables users to select the desired process and presents all findings in a structured manner.

## 4 METHODOLOGY

As discussed earlier, the framework comprises a Passive Monitoring Node realized on an FPGA for non-intrusive signal observation, a Firmware Acquisition Node - FirmEx, for automated firmware acquisition, and a Processing Node - Raspberry Pi, that handles processing, verification, and visualization. The following sections describe the design and role of each of these nodes in the overall system.

## 4.1 Sniffer Design (Verilog)

Hardware protocol sniffers are generally designed to passively monitor communication buses such as $I^2C$, SPI, UART, and CAN. Their primary goal is to capture essential elements of bus activity, such as control signals, addresses, and data frames, without interfering with the ongoing communication. The design principles behind such sniffers are largely generic and can be adapted across different protocols with protocol-specific decoding logic.

In this work, the focus is placed on $I^2C$, as the experimental setup involved an Arduino UNO communicating with an MPU-6500 sensor over an $I^2C$ bus. Accordingly, an $I^2C$ sniffer is implemented in Verilog to capture key features of the communication, including start conditions, slave addresses, read/write (R/W) bits, register addresses, and data bytes. The implementation allowed real-time observation of traffic while remaining non-intrusive to the master–slave communication.

The core of the design is a Finite State Machine (FSM) that continuously monitors the SDA and SCL lines of the $I^2C$ bus. The FSM is structured to identify and decode specific protocol phases in sequence:

- **Start Condition Detection:** The FSM transitions to the capture phase upon detecting a falling edge on SDA while SCL is high, which signifies a start condition.
- **Slave Address Capture:** The FSM captures the first byte following the start condition as the slave address, including the R/W bit.
- **Register Address Capture:** The next byte is interpreted as the internal register address targeted by the master.
- **Data Byte Handling:** Subsequent bytes are treated as data payloads. These may be written to the slave or read from it, depending on the R/W bit.
- **Stop Condition Detection:** A rising edge on SDA while SCL is high is interpreted as a stop condition, signaling the end of a transaction.
- **Repeated Start Condition Detection:** The FSM also detects repeated start conditions, which occur mid-transaction without a preceding stop. In such cases, the FSM resets appropriately to begin capturing a new address phase while preserving context.

Each byte is sampled on the rising edge of the SCL signal to ensure reliable timing in accordance with the $I^2C$ standard.

The sniffer is designed to work with both read and write transactions, and supports multi-byte sequences. It does not drive any lines, ensuring non-invasive behavior on the bus.

The framework is implemented in Verilog, a hardware description language (HDL), using the Xilinx Vivado Design Suite. The module is synthesized and deployed on a Basys 3 development board containing a Xilinx Artix-7 FPGA. The implementation is also verified with an LM75 temperature sensor, BME280 environmental sensor (temperature, pressure, humidity), ADS1115 analog-to-digital converter and DS3231 real-time clock.

## 4.2 UART Integration and Data Logging

A UART module is interfaced with the sniffer to enable data transfer from the FPGA to a host PC, allowing captured bus activity to be observed and analyzed externally.

For this implementation, the UART module is designed as a separate Verilog block to handle serial communication with the host PC. It interfaces directly with the $I^2C$ sniffer module through a simple handshaking protocol based on a flag setting and clearing mechanism

Whenever the $I^2C$ sniffer completes the capture of a valid transaction—such as a register read or write—it raises a signal indicating that a valid and complete set of bytes has been captured. Once this flag is received, the UART FSM initiates the transmission

The UART module contains a finite state machine (FSM) that includes states for:

- **Slave Address Transmission:** Sends the 7-bit slave address concatenated with the read/write (R/W) bit.
- **Register Address Transmission:** Sends the address of the internal register accessed by the master.
- **Data Byte Transmission:** If present, sends the data byte either read from or written to the slave device.

This modular separation ensures that the $I^2C$ sniffer and UART operate asynchronously, with minimal coupling, and makes it easier to scale or modify individual modules in the future. All data captured by the sniffer is ultimately transmitted to the host PC via UART, where it can be visualized in a serial terminal. Optionally the transmitted data can also be logged through the serial terminal for offline analysis.

The integration is synthesized on the Basys 3 FPGA board and validated using live $I^2C$ traffic from sensors such as the MPU-6500 and LM75.

## 4.3 ML-Based Anomaly Detection (Z-Score Model on RPi)

The Raspberry Pi receives real-time sensor data and applies a lightweight statistical anomaly detection algorithm based on Z-score analysis. During an initial training phase, a buffer of valid readings is collected to compute the baseline mean and standard deviation of the observed signal. Once trained, incoming values are continuously evaluated against this learned distribution. If a new data point exceeds a configurable deviation threshold (e.g., 2.5 standard deviations from the mean), it is flagged as anomalous.

The logic behind our anomaly detection is simple but effective. We use a statistical method called the Z-score, which helps determine how far a new sensor reading deviates from what is considered normal. During the training phase, the Raspberry Pi collects a buffer of normal sensor readings and calculates the mean ($\mu$) and standard deviation ($\sigma$) of this data. Once trained, each new reading $x_i$ is evaluated using the following formula:

$$Z_i = \frac{x_i - \mu}{\sigma}$$

If the absolute value of $Z_i$ exceeds a certain threshold (e.g., $|Z_i| > 2.5$), the reading is flagged as anomalous. This threshold helps strike a balance between sensitivity and false positives.

The advantage of this method is that it adapts gradually over time, since the buffer is constantly updated with recent normal values. The model stays in sync with slow environmental drifts. At the same time, it remains responsive to short-term changes that could indicate real anomalies. During training, we ignore abnormal jumps between two consecutive readings and flag them separately

**Algorithm 1:** Multi-Sensor Anomaly Detection with Z-Score and Spike Filtering

**Input:** Sensor name $S_n$, serial port $P$, baud rate $B$
**Output:** Real-time classification of sensor values as Normal, Anomaly, Spike, or Training

**Sensor Configurations:**
- Sensor $X$: parser = PARSER_X, $\tau = \tau_X$, $T_{\text{train}} = T_X$, $S = S_X$, indices = $(i_X, j_X)$, spike_jump = $\Delta_X$

Initialize detector with $(\tau, T_{\text{train}}, \Delta)$ from chosen sensor $S_n$ ;
Open serial connection on $(P, B)$ ;
Initialize $prev\_value \leftarrow \varnothing$, $cooldown \leftarrow 0$ ;
**while** *connection is active* **do**
  Read $S$ bytes into $packet$ ;
  **if** $|packet| = S$ **then**
    Extract bytes $(i, j)$ from $packet$ according to sensor config ;
    $value \leftarrow \text{PARSER}(packet[i], packet[j])$ ;
    $status \leftarrow \text{DETECTOR.ADDSAMPLE}(value)$ ;
    `// Spike filtering`
    **if** $prev\_value \neq \varnothing$ ***and*** $|value - prev\_value| > \Delta$ **then**
      $cooldown \leftarrow C$ ;
      Output "Spike detected: $prev\_value \rightarrow value$" ;
    **else if** $cooldown > 0$ **then**
      Output "Cooling down... Value: $value$" ;
      $cooldown \leftarrow cooldown - 1$ ;
    **else**
      **if** $status = Anomaly$ **then**
        Output "Anomaly detected! Value: $value$" ;
      **else if** $status = Normal$ **then**
        Output "Normal Value: $value$" ;
      **else**
        Output "Training... Value: $value$" ;
    $prev\_value \leftarrow value$ ;

**Function** `Parser_X(high, low)`**:**
  $raw \leftarrow (high \ll 8) \mid low$ ;
  Apply transformation specific to sensor $X$ ;
  **return** scaled value ;

post-training. Such spikes are considered transmission artifacts often caused by UART noise, packet alignment issues, or timing mismatches between sensor data capture and UART dispatch.

This approach balances simplicity, real-time responsiveness, and resilience to hardware-level noise. The model runs entirely on the Raspberry Pi without GPU acceleration or external dependencies, making it suitable for embedded environments with limited compute resources. The overall design allows the model to run in real time, detect subtle intrusions, and handle all such low-level disturbances gracefully.

## 4.4 Firmware Extraction via FTDI-Based Board

Firmware extraction starts by identifying accessible debug or memory access points such as test pads, headers, or flash chips on our target devices. Interfaces like SPI can be accessed using SOIC clips, while others like UART or JTAG may be reached through jumper wires.

To support this, we developed a custom USB interface board, FirmEx, based on the FT2232H chip (2). It consolidates multiple protocols onto a single platform and includes level shifters with a selectable voltage rail (1.8V, 3.3V, 5.0V, or VTGT) to ensure electrical compatibility with different targets. The design is heavily inspired by the open-sourced documentations of Tigard[18].

The EEPROM is configured with Vendor ID 0x0403 and Product ID 0x6010. Channel A is mapped to UART (VCP driver) and Channel B to MPSSE (D2XX driver). This allows direct compatibility with tools like flashrom, openocd, and pyftdi.

The configuration files are intentionally minimal and interface-specific. They enable hardware initialization while abstracting away low-level scan logic. All high-level orchestration, including tool invocation, data parsing, and error handling, is managed within the Python script.

## 4.5 Voltage and Protocol Switching Logic

To handle the diversity of embedded interfaces, FirmEx includes a dual-switch configuration system that simplifies both electrical compatibility and logical routing. The first switch controls voltage level translation by routing power either from the board's internal regulator or the target system. The second switch determines the logical wiring configuration, enabling support for interface variants that share overlapping physical lines.

For example, due to the shared signaling characteristics of certain communication protocols, specific headers are designed to support multiple interfaces. The same header may carry both data and clock lines that serve one protocol in one switch position and another in a different position. As a result, protocols like I²C can be reconfigured onto headers nominally assigned for SPI, depending on user selection.

This multiplexed design minimizes the number of physical headers while maximizing protocol coverage. It also simplifies cable management and reduces board complexity, which is particularly beneficial in scenarios requiring rapid interface switching or repeated testing across multiple devices.

LED indicators provide immediate feedback on power status and active configuration modes, aiding in troubleshooting and setup verification. Together, these features make the board highly adaptable and user-friendly for firmware acquisition tasks across a broad spectrum of embedded platforms.

## 4.6 Firmware Integrity Verification

Once firmware is extracted, a SHA-256 hash of the image is computed and compared against a reference dataset to check for known firmware versions. This dataset is sourced from a prior study [31] on embedded system vulnerabilities and contains approximately 11,000 firmware hashes (mainly routers) collected from devices such as routers, cameras, and network appliances. The database is used to identify extracted images that matched known firmware,

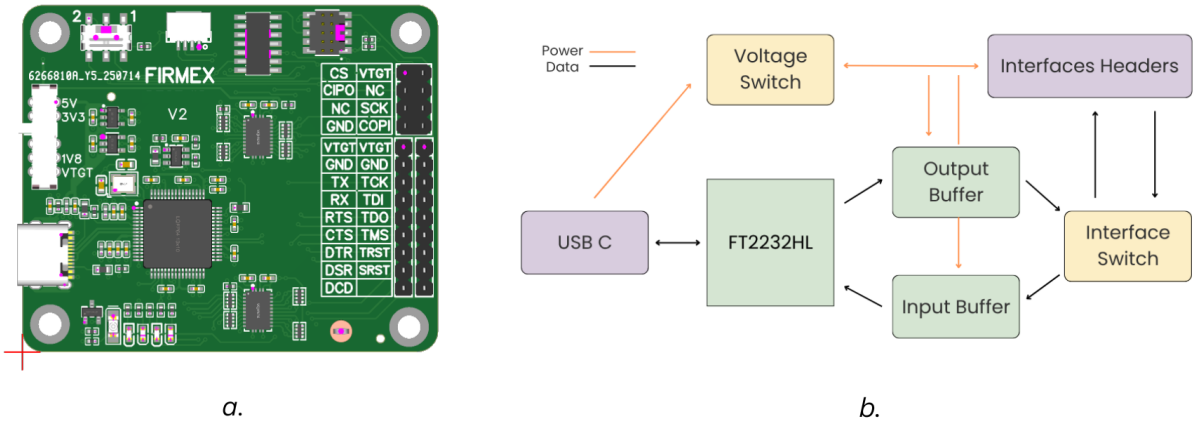


**Figure 2: a. FirmEx Board: Custom Designed FT2232H Multi-interface Firmware Extraction Tool b. Design Logic used for FirmEx**

allowing quick correlation with associated vulnerabilities or known behaviors. We aim to enhance the database as our Future Work extending it to a wider range of devices.

## 5 ATTACK SIMULATION AND SECURITY ANALYSIS

### 5.1 FPGA-Based Surveillance of Sensor Bus

To evaluate security threats in sensor-based embedded systems, a controlled experimental testbed is built around the FPGA-based I²C sniffer. The Arduino Uno is configured as the I²C master and connected directly to the sensor behaving as the slave The FPGA is physically inserted to the same SDA and SCL lines as the master and slave on the I²C bus. This configuration enables passive interception of the I²C traffic while ensuring that the natural operation of the bus remains unaffected.

The FPGA captures low-level bus events such as start/stop conditions, slave addresses, register addresses, acknowledgments, and data payloads. Captured transactions are buffered and streamed over UART to a host PC, where a processing node records the logs for further inspection. This architecture provides real-time visibility into sensor communication without introducing delays or modifying bus timing.

### 5.2 Firmware Modification Scenarios

To validate the framework's ability to detect firmware tampering, the original firmware is first extracted from the target device and stored as a cryptographic golden reference. Following this, the device's flash is intentionally overwritten with altered binaries.

After reflashing, the modified firmware is re-extracted and hashed using SHA-256. These hashes are compared against the reference values from the database of firmware hashes mentioned earlier, and any mismatch is flagged as an integrity violation. This approach effectively revealed even subtle modifications, without requiring access to source code or internal documentation.

### 5.3 Anomaly Flagging and Logging

Sensor data from the I2C bus is continuously fed into the anomaly detection module running on the Raspberry Pi. The trained Z-score model evaluated statistical deviations from baseline behavior. The system logged:

- Statistically significant deviations from the trained distribution
- Sequences that cross adaptive thresholds
- Spike events caused by UART noise or frame desynchronization, which are filtered but logged separately

### 5.4 Comparative Evaluation of Anomaly Detection Methods

To assess the suitability of different anomaly detection approaches for embedded runtime monitoring, we compared Z-score–based statistical detection with a machine learning based Isolation Forest model which is widely used for anomaly detections we're interested in. Both models are tested on live temperature sensor data across two independent experimental runs. In each experiment, anomalies are injected in real time through a separate fault injection module, introduced to simulate real-world runtime deviations.

Figure 3 shows the result of a session where both genuine anomalies and serial noise–induced spikes were present. While Isolation Forest successfully identified only the most extreme outliers (e.g., corruption-induced 144°C spikes), it failed to flag smaller but meaningful deviations. Z-score, in contrast, detected all subtle variations caused by sensor manipulation. Figure 4 shows a second, spike-free run focused on fine-grained changes. The Z-score model again outperformed Isolation Forest in flagging temperature drift caused by fault injection events. This confirms that Z-score–based methods are better suited for sensitive anomaly detection in embedded trust contexts.

When such anomalies are detected, the system logged the event with metadata including timestamp, sensor source, and deviation score. A rolling buffer of recent events is maintained to allow retrospective analysis and model fine-tuning. Anomalies are flagged

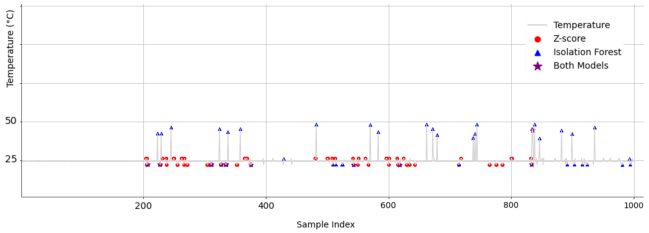


**Figure 3: Full-scale comparison of Z-score and Isolation Forest. Z-score detects subtle drifts; Isolation Forest flags only extreme spikes.**

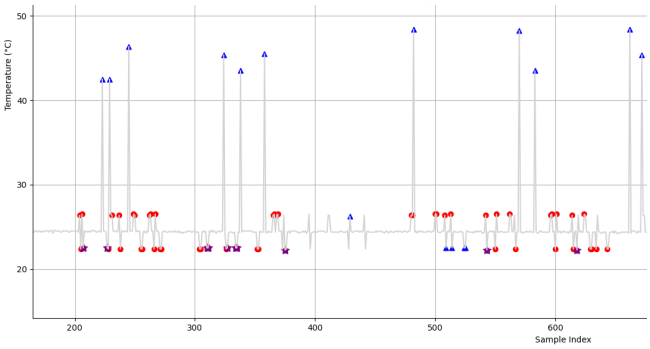


**Figure 4: Zoomed-in view showing Z-score's sensitivity to small runtime deviations. Isolation Forest misses these anomalies.**

without halting the system or interrupting communication, preserving passive monitoring behavior.

## 5.5 Visual Verification with GUI

A user-friendly GUI is developed on the processing node to provide real-time visual feedback. The interface included:

- Live I²C bus activity logs
- A status dashboard for anomaly alerts
- A firmware integrity check module showing hash comparison results
- User controls to initiate extraction via JTAG, SPI, or UART
- A sensor selection panel allowing the user to choose which connected sensor is used for anomaly detection

Color-coded indicators is used to show anomaly severity and firmware mismatch status. The GUI allowed operators to toggle between interfaces and observe results without needing command-line interaction. This visual feedback is essential for quick validation in lab tests and demonstrations.

# 6 RESULTS

## 6.1 I²C Traffic Logs & FSM Visualization

Simulation results confirmed that the FSM correctly handled I²C state transitions and captured complete byte sequences without loss. The timing of the sampled edges matched the expected SCL/SDA relationships cycle-for-cycle, validating the correctness of the clocking and transition logic.

During hardware testing on the Basys 3 FPGA, the sniffer successfully intercepted live I²C traffic between the Arduino UNO master and a set of four different slave devices. Over a large number of I²C transactions, the system consistently captured slave addresses, register addresses, and data bytes without missing or corrupted frames observed.

Once I²C transactions were captured without errors, the UART module was integrated to facilitate real-time monitoring and offline analysis via data logging. The captured transactions were serialized and transmitted to a host PC, with each transaction comprising the slave address, register address, and data byte. Validation of the UART transmission module was carried out using RealTerm and HTerm, both of which support hexadecimal display modes that simplified verification of transmitted values and framework correctness. Furthermore, the serial monitors provided a logging option to directly store traffic into .txt files on the host PC, enabling later analysis or post-processing of I²C communication traces.

Finally, fault injection experiments confirmed that anomalies introduced on SDA and SCL lines (such as clock stretching and stuck-low conditions) were consistently reflected in the captured transactions.

## 6.2 ML Detection Accuracy and Precision

Anomalies were injected in real time through a dedicated fault injection module , while ensuring timestamps were recorded for validation. Performance was evaluated across 1000 samples using annotated ground truth labels. The Z-score–based detector yielded the following:

- **True Positive Rate (Recall)**: 67.8%
- **Precision**: 94.73%
- **False Positive Rate**: 0.004%

Most false positives were short-lived and self-corrected after the sliding window buffer adjusted to the new baseline. The Z-score model proved sensitive enough to detect timing deviations, physical intrusions, and unexpected temperature drifts while maintaining low false alarm rates. In contrast, the Isolation Forest model detected only extreme outliers, showing poor recall in real-world subtle intrusion scenarios.

## 6.3 Comparative Overview of Firmware Extraction / Integrity Outputs

Table 1 summarizes our per-device extraction results across 19 targets (18 successful extractions, 1 failure).

**High-level summary.** Across the 18 devices with successful dumps the automated FirmEx pipeline reduced end-to-end extraction and verification time substantially: the mean manual extraction time is ≈ 91.0 s (1 min 31 s) while the mean FirmEx time is ≈ 41.1 s, giving a mean speedup of ≈ 2.47× (FirmEx saves ∼ 59% of operator time on average). Times are heterogeneous (median manual ≈ 32.5 s; median FirmEx ≈ 15.5 s) because a small number of large-flash devices dominate the tail. One device (Amazon Echo) failed due to strong protection; several routers with large external flash show the smallest relative gains because raw transfer is the bottleneck.

**Integrity / verification outcomes.** For devices that produced readable images, FirmEx computes a SHA-256 digest and compares it against our reference corpus. Successful dumps are recorded as "Extracted & Verified" or "Extracted"; where an extracted image

**Table 1: Survey of Firmware Extraction Across 19 Devices**

| Category | Device | JTAG/SWD | SPI | UART | I2C | Manual Time[1] | FirmEx Time[2] | Verification/Dump Status | Obfuscation |
|---|---|---|---|---|---|---|---|---|---|
| **Dev Boards** | Tiva C series | ✓ | | | | 30 s | 5 s | Extracted & Verified | Simple |
| | BeagleBoard Rev C4 | ✓ | | | | ∼5 min | ∼1 min | Extracted | Simple |
| **IoT / Smart Home** | TP-Link Tapo C100 Camera | | ✓ | | | 34 s | 15 s | Extracted & Verified | Moderate |
| | Crow Runner Alarm | | | | ✓ | 31 s | 16 s | Extracted | Simple |
| | Nintendo Alarmo Clock | ✓ | | | | ∼6.5 min | ∼3.2 min | Extracted | Moderate |
| | WeMo Link | | | ✓ | | 29 s | 13 s | Extracted | Simple |
| | Amazon Echo | | | | | ∼26 min | - | Failed | Hard |
| **Research** | AVR ATtiny85 | | | ✓ | | 12 s | 5 s | Extracted | Simple |
| | NAE-CW308T (ChipWhisperer) | ✓ | | | | 12 s | 6 s | Extracted & Verified | Simple |
| **Routers** | TP-Link Archer C7 | | ✓ | | | 22 s | 8 s | Extracted & Verified | Moderate |
| | TP-Link TL-WR1043N | | ✓ | | | 14 s | 6 s | Extracted & Verified | Moderate |
| | D-Link DIR-615 | | ✓ | | | 8 s | 4 s | Extracted & Verified | Moderate |
| | Netgear R7000 Nighthawk | | | ✓ | | 58 s | 30 s | Extracted & Verified | Moderate |
| | Netgear AC1750 | | | ✓ | | 44 s | 21 s | Extracted & Verified | Moderate |
| | Asus RT-AC68U | | ✓ | | | ∼1.1 min | 48 s | Extracted & Verified | Moderate |
| | AVM FRITZ!Box 7490 | | ✓ | | | 12 s | 5 s | Extracted & Verified | Moderate |
| | Ubiquiti EdgeRouter ER-X | | ✓ | | | ∼3.8 min | ∼2.1 min | Extracted & Verified | Hard |
| | MikroTik RB750Gr3 | | ✓ | | | ∼3.2 min | ∼1.8 min | Extracted & Verified | Hard |
| | Linksys WRT1900AC | | ✓ | | | ∼2.6 min | ∼1.2 min | Extracted & Verified | Hard |
| **Total (# / %)** | 19 Devices | 4 / 21% | 9 / 47% | 4 / 21% | 1 / 0.05% | 18/19 Extracted (95%), 2/19 Strong Protection (10%) | | | |

[1] Manual Time denotes the time taken by a human operator to extract the firmware or extract & verify using traditional hardware debugging methods. [2] FirmEx Time denotes the time taken by the automated FirmEx framework to complete extraction or extraction & verification. *Note: All timings (in seconds) were measured with cables pre-connected.*

differs from known hashes we record it as a mismatch and retain both images and logs for manual analysis.

**Conclusion.** The per-device results in Table 1 demonstrate that automated, multi-protocol orchestration substantially reduces operator time and increases repeatability for a broad class of embedded targets (typical speedup $\approx 2.5\times$).

However, hardware-enforced protections and on-chip encryption remain a hard boundary for non-invasive workflows.

We discuss more about this here [*A.2] in detail.*

## 7 CONCLUSION AND FUTURE WORK

This work presented FirmEx, a unified framework that integrates passive sensor monitoring with automated firmware acquisition and verification to strengthen trust in embedded devices. The framework shows that lightweight, interpretable methods such as an adaptive Z-score detector, when carefully engineered with spike filtering and rolling baselines, can reliably capture subtle deviations that signal potential tampering. Coupled with a multi-protocol acquisition board and automated hashing, FirmEx streamlines what is otherwise a fragmented and error-prone process, reducing operator effort while improving consistency and auditability. Evaluation on a diverse set of real devices confirmed that this combination of simplicity, efficiency, and automation can provide timely, actionable insights into both runtime anomalies and firmware integrity. Overall, FirmEx demonstrates that effective embedded security does not require heavyweight machine learning or invasive instrumentation, but can be achieved through the thoughtful integration of lightweight statistical methods and pragmatic hardware support.

As a next step, the proposed framework can be evolved into a fully integrated custom hardware solution featuring a dedicated processor, replacing the modular FPGA–Raspberry Pi–FTDI setup. Such a board would offer tighter coupling between firmware extraction, anomaly detection, and verification processes, making it more suitable for industrial or field deployments where reliability, compactness, and automation are essential. Currently, raw flash dumps may include bootloaders, configuration data, or padding alongside the actual application firmware, which can distort direct hash comparisons. To improve precision, the future work should incorporate automated binwalks or custom binary parsers to isolate relevant firmware sections, extract meaningful regions, and then compute hashes only on those segments.

*We discuss more about the conceptual/high-level implementation of the future work here [A.3] in detail.*

# REFERENCES


[1] Yang Su and D. C. Ranasinghe, "Leaving Your Things Unattended is No Joke! Memory Bus Snooping and Open Debug Interface Exploits," *arXiv preprint arXiv:2201.07462* (2022).

[2] E. DeBusschere and M. McCambridge, "Modern Game Console Exploitation," University of Arizona (2012). [Online]. Available: https://www2.cs.arizona.edu/~collberg/Teaching/466-566/2012/Resources/presentations/2012/topic1-final/report.pdf

[3] S. U. Haq, Y. Singh, A. Sharma, R. Gupta, and D. Gupta, "A survey on IoT & embedded device firmware security: architecture, extraction techniques, and vulnerability analysis frameworks," *Discover Internet of Things* (2023).

[4] F. Gomez-Bravo, R. Jimenez Naharro, J. Medina Garcia, J. Gomez Galan, and M. S. Raya, "Hardware Attacks on Mobile Robots: I2C Clock Attacking," in *Robot 2015: Second Iberian Robotics Conference* (2016)

[5] L. Verderame, A. Ruggia, and A. Merlo, "PARIOT: Anti-Repackaging for IoT Firmware Integrity", *arXiv preprint arXiv:2109.04337* (2023)

[6] S. Falas, C. Konstantinou, and M. K. Michael, "A Modular End-to-End Framework for Secure Firmware Updates on Embedded Systems", *arXiv preprint arXiv:2007.09071* (2021)

[7] M. Rodler, C. Wressnegger, and K. Rieck, "BootKeeper: Validating Software Integrity Properties on Boot Firmware Images," *arXiv preprint arXiv:1903.12505* (2019).

[8] A. Chatterjee and B. S. Ahmed, "IoT Anomaly Detection Methods and Applications: A Survey," *arXiv preprint arXiv:2207.09092* (2022).

[9] Vasile, S., Oswald, D., Chothia, T. (2019). Breaking All the Things—A Systematic Survey of Firmware Extraction Techniques for IoT Devices. In: Bilgin, B., Fischer, J.B. (eds) Smart Card Research and Advanced Applications. CARDIS 2018. Lecture Notes in Computer Science, vol 11389. Springer, Cham. https://doi.org/10.1007/978-3-030-15462-2_12

[10] M. Guri, Y. Poliak, B. Shapira, and Y. Elovici. 2015. *"JoKER: Trusted Detection of Kernel Rootkits in Android Devices via JTAG Interface"*. In *Proceedings of the 2015 IEEE Trustcom/BigDataSE/ISPA* (2015).

[11] "SWD Vs. JTAG: A Comparison of Embedded Debugging Interfaces," *ReversePCB* (2023). [Online]. Available: reversepcb.com/swd-vs-jtag

[12] A. Parsa, "'Accessing and Dumping Firmware Through UART," *CyberArk Threat Research Blog*. [Online]. Available: cyberark.com/resources/threat-research-blog/accessing-and-dumping-firmware-through-uart

[13] "Dumping SPI Flash Memory of Embedded Devices," *NSIDE ATTACK LOGIC Tech Blog*. [Online]. Available: nsideattacklogic.de/en/dumping-spi-flash-memory-of-embedded-devices-2/

[14] "IoT Security – Part 16 (101 – Hardware Attack Surface: I²C)," *Payatu IoT Masterclass* [Online]. Available: payatu.com/masterclass/iot-security-part-16-101-hardware-attack-surface-i2c/

[15] "Winning The Interface War: Extracting Information From Electronic Devices With The SEC Xtractor," SEC Consult Blog [Online]. Available: https://sec-consult.com/blog/detail/winning-the-interface-war-extracting-information-from-electronic-devices-with-the-sec-xtractor

[16] Tenet Technetronics, "Attify Badge – UART, JTAG, SPI, I2C," Tenet Technetronics, [Online]. Available: https://www.tenettech.com/product/attify-badge-uart-jtag-spi-i2c

[17] "Bus Pirate," [Online]. Available: https://buspirate.com/

[18] Tigard: Multi-Protocol Hardware Hacking Tool. Available at: https://github.com/tigard-tools/tigard

[19] Total Phase, Inc., "Beagle I²C/SPI Protocol Analyzer." Available: https://www.totalphase.com/products/beagle-i2cspi/ (accessed Aug. 25, 2025).

[20] Saleae, Inc., "Saleae Logic Analyzers." Available: https://www.saleae.com/ (accessed Aug. 25, 2025).

[21] S. Checkoway, D. McCoy, B. Kantor, D. Anderson, H. Shacham, and S. Savage, "Comprehensive Experimental Analyses of Automotive Attack Surfaces," in *Proceedings of the USENIX Security Symposium (USENIX Security '11)*, 2011. Available: https://www.usenix.org/conference/usenixsecurity11/comprehensive-experimental-analyses-automotive-attack-surfaces

[22] M. T. R. Laskar, J. Huang, V. Smetana, C. Stewart, K. Pouw, A. An, and S. Chan, "Extending Isolation Forest for Anomaly Detection in Big Data via K-Means," 2021. [Online]. Available: https://arxiv.org/abs/2104.13190

[23] M. Nadeski, "Bringing Machine Learning to Embedded Systems," *Texas Instruments White Paper* (2019).

[24] W. Roth, G. Schindler, B. Klein, R. Peharz, S. Tschiatschek, H. Fröning, F. Pernkopf, and Z. Ghahramani, "Resource-Efficient Neural Networks for Embedded Systems," *arXiv preprint arXiv:2001.03048* (2020).

[25] J. Wright and J. Cache, *Hacking Exposed Wireless: Wireless Security Secrets and Solutions*, 2nd ed. McGraw-Hill, 2010.

[26] S. van Ieperen, "Sniffing Communications Between an Arduino and Its Peripheral Sensor," *Twente Student Conference on IT*, 2021.

[27] A. Tevesz, "Non-invasive I2C Hardware Trojan Attack Vector," in *IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, 2022.

[28] M. Chhetri et al., "Hardware Attacks on Mobile Robots: I2C Clock Attacking," *IEEE Access*, vol. 10, pp. 123456–123465, 2022.

[29] A. Jha, "IoT Security - Part 16 (101 - Hardware Attack Surface: I2C)," Payatu, 2020. [Online]. Available: https://payatu.com/masterclass/iot-security-part-16-101-hardware-attack-surface-i2c

[30] A. Tevesz, "Hardware Hacking 101 – E01: I2C Sniffing," CUJO AI Blog, 2020. [Online]. Available: https://cujo.com/blog/hardware-hacking-101-e01-i2c-sniffing/

[31] R. Helmke, E. Padilla, and N. Aschenbruck, "Mens Sana In Corpore Sano: Sound Firmware Corpora for Vulnerability Research," in *Proceedings of the Network and Distributed System Security (NDSS) Symposium 2025* (2025), https://dx.doi.org/10.14722/ndss.2025.230669

[32] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)* (2008).

[33] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.* (2009).

[34] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics* (1962).

[35] Espressif Systems, "ESP32 Security Guide — Flash Encryption," Espressif technical documentation (2020).

[36] G. Farrelly, M. Chesser, and D. C. Ranasinghe, "Ember-IO: Effective Firmware Fuzzing with Model-Free Memory Mapped IO," in *Proceedings of the ACM ASIA Conference on Computer and Communications Security (ASIA CCS '23) (2023). Available: https://doi.org/10.1145/3579856.3582840

[37] M. Armanuzzaman, A.-R. Sadeghi, and Z. Zhao, "Building Your Own Trusted Execution Environments Using FPGA (BYOTee)," in *Proceedings of ACM ASIA CCS* (2024). doi:10.1145/3634737.3637644.

[38] L. Delshadtehrani, A. Joshi, A. Zhang, A. H. Khalili, J. A. Dunlap, et al., "PHMon: A Programmable Hardware Monitor and its Security Use Cases," in *Proc. USENIX Security* (2020).

[39] M. Zhao, M. Gao, and C. Kozyrakis, "ShEF: Shielded Enclaves for Cloud FPGAs," in *ASPLOS* (2022).

[40] S. Pereira, D. Cerdeira, C. Rodrigues, and S. Pinto, "Towards a Trusted Execution Environment via Reconfigurable FPGA (TEEOD)," arXiv:2107.03781 (2021).

# A APPENDIX

## A.1 Isolation Forest: overview and why it underperformed in our setting

Isolation Forest (iForest) is an ensemble, tree-based, unsupervised method that detects anomalies by *isolating* observations via random recursive partitioning [32, 33]. Instead of explicitly modelling the normal data distribution, iForest uses the observation that "anomalies" are easier (require fewer random splits) to isolate than normal points.

*Core definitions.* Consider a dataset of size $n$. An isolation tree (iTree) is constructed by recursively selecting a feature and a split value uniformly at random and partitioning the data until each instance is isolated. The path length $h(x)$ of instance $x$ in an iTree is the number of edges traversed from the root to the external node that contains $x$. A forest is a collection of $t$ iTrees, typically trained on subsamples of size $\psi \leq n$. The average path length across the forest is $E[h(x)]$.

The iForest anomaly score for instance $x$ (normalized by the expected path length of unsuccessful random partitions) is defined as:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}},$$

where $c(n)$ is the average path length of unsuccessful searches in Binary Search Trees and can be approximated as

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n},$$

with $H(m)$ the $m$-th harmonic number (and in large-$m$ limit $H(m) \approx \ln(m) + \gamma$ where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant) [32]. Points with $s(x, n)$ near 1 are highly anomalous; points with $s(x, n) \approx 0.5$ are typical.

*Computational characteristics.* An iForest has favorable asymptotic properties (linear in the training set size under sub-sampling) and low memory usage, because each tree is grown on a small $\psi$-sized sample and trees are shallow on average [32]. Typical complexity for building a forest with $t$ trees and sample size $\psi$ is roughly $O(t \cdot \psi \log \psi)$ for construction and $O(t \log \psi)$ per query (path traversal per tree).

*Why iForest underperformed for FirmEx (summary).* While iForest is powerful in many anomaly-detection contexts, the FirmEx monitoring workload differs in ways that reduce iForest's practical benefit:

- **Low dimensionality / streaming one-feature signals:** Our sensors and bus-derived metrics are typically low-dimensional (often effectively univariate for a single detector). In such cases, simple statistical tests (e.g., rolling Z-score) are already highly discriminative and far cheaper to compute [33].
- **Runtime & memory constraints:** Building and maintaining many trees (even on sub-samples) increases latency and memory footprint on our Raspberry Pi-class processor, which matters when detection latency must be low (tens to a few hundreds of milliseconds).

| Property | Isolation Forest | Rolling Z-score (FirmEx) |
|---|---|---|
| Per-sample compute cost | moderate ($t \cdot \log \psi$) | **very low** (O(1)) |
| Memory footprint | moderate (forest storage) | negligible (running mean/var) |
| Training requirement | offline/subsampled | none (online Welford updates) |
| Sensitivity to small drifts | limited | **high** |
| Typical FPR (our deployment) | higher | **lower** |
| Interpretability | medium | **high** |

**Table 2: Comparison of Isolation Forest and rolling Z-score detector.**

- **Sensitivity-to-small-sustained-drifts:** iForest isolates points that are structurally separated from the bulk of the data; sustained small drifts (adversarial slow tampering) may not significantly change tree path lengths but will change running mean/std quickly making Z-score more sensitive to tampering patterns we care about.
- **False positives from noisy bursts:** In our experiments iForest produced more false positives on transient but benign communication glitches; our Z-score + spike-filter pipeline achieved lower FPR with comparable recall.

*Implementation notes.* For streaming deployment we used a rolling baseline maintained via Welford's online algorithm for mean and variance (numerically stable, constant memory and CPU) and used a configurable threshold $T$ on $|Z|$; a separate spike filter pre-processes obvious transport artifacts. Where computational resources are available and multi-dimensional signals are jointly informative, iForest or compact autoencoders can be revisited; however, for FirmEx's target operating point the rolling Z-score gave the best trade-off between latency, precision, and interpretability [33, 34].

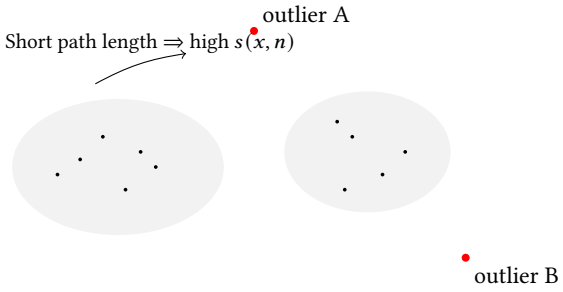


**Figure 5: Illustration: two dense clusters (normal) and two isolated points (anomalies). Isolation Forest isolates red points with shorter average path length across trees, producing higher anomaly scores.**

## A.2 Firmware extraction: practical limitations and failure modes

This section summarizes the most common and reproducible reasons firmware extraction fails or yields unusable data. We focus on the practical symptoms, root causes, and short guidance on diagnosis or mitigation so that reviewers and future users of FirmEx can understand the boundaries of automated acquisition workflows.

*High-level taxonomy.* Firmware extraction attempts generally use one of three vectors: **(i)** on-chip debug access (JTAG, SWD) that reads memory via the CPU's debug-bridge; **(ii)** direct flash access (SPI/NOR, NAND) via a programmer; and **(iii)** bootloader or UART boot interfaces that allow reading or reflashing via serial protocols. Each has its own failure modes, summarized below.

*Common failure modes.*

- **Read-out protection / locked debug:** Many MCUs (ARM Cortex family, others) support security bits or fuses that disable debug read-out or require vendor authentication. When enabled, OpenOCD or other debug tools may connect but then refuse to read memory (error messages such as "target state unknown / cannot read memory" are typical). These protections are sometimes irreversible without hardware-level fuse resets or vendor keys.
- **On-chip flash encryption:** Modern SoCs (e.g., many Espressif chips, some vendor SoCs) support full-flash encryption. The raw SPI/NOR dump will therefore be ciphertext; decryption occurs only inside the chip when the SoC's internal key material is present. Without key recovery (side-channel or vendor keys) the dump is unusable. Attempting to hash or carve plaintext from an encrypted dump will produce gibberish. [35]
- **Protected / authenticated bootloaders:** Some devices require an authentication handshake in the bootloader before it exposes read or reflashing commands. If the handshake fails, the bootloader returns an error or a limited interface. Reflashing or reading via UART bootloaders may thus be impossible without vendor cooperation.
- **Memory-mapped peripherals vs. firmware:** Some targets map peripheral or configuration storage into the address space. Blindly dumping an MMIO range can return stale sensor data, FIFOs, or device registers, not executable firmware. Proper extraction requires identifying flash address ranges vs MMIO. [36]
- **External flash with hardware protection:** External SPI/NOR chips may use quad/dual I/O, require specific voltage rails, or be protected by GPIO-driven hold/ WP pins. Incorrect wiring or voltage can yield read failures or corrupt reads.
- **Non-trivial image formats (packing/encryption/compression):** Even when raw flash is readable, firmware images are frequently compressed, packed, or wrapped in vendor-specific containers. Tools such as `binwalk` and manual reverse engineering are often required to extract the executable region used for hash comparison.

*Practical advice for reproducibility.*

- Always identify the CPU and memory map (vector table, boot ROM) where possible; this helps distinguish flash ranges from MMIO. If the vendor provides a reference '.map' or '.elf' in firmware packages, use it to locate text/data sections prior to hashing. [7]
- When a dump looks like random bytes, check known-encryption markers or vendor docs (e.g., Espressif flash encryption options). Do not assume a failed carve equals a failed dump. [35]
- Preserve logs (OpenOCD, flashrom output) — they often contain detection errors that explain failures (voltage mismatch, unsupported chip). Include sanitized logs in the artifact bundle for reviewers.

| Failure mode | Symptom | Likely cause | Practical mitigation |
|---|---|---|---|
| JTAG/SWD protected | OpenOCD halts, read commands fail | Debug fuses / read-out disable | Check vendor datasheet; try vendor-assisted unlock; otherwise mark as non-recoverable |
| Encrypted flash | Read bytes look random; no ELF/SquashFS signatures | On-chip flash crypto or encrypted firmware image | Look for vendor encryption features; consider key-recovery or side-channel (out of scope) |
| Bootloader auth required | Bootloader denies dump/reflash commands | Authenticated boot protocol | Record handshake; if vendor key unavailable, cannot proceed safely |
| MMIO vs flash confusion | Dump contains registers, FIFO contents or short repetitive data | Wrong address range or memory map error | Use datasheet / .map file; halt CPU and read vector table to locate flash base |
| Physical SPI issues | Partial reads, CRC errors | SOIC clip misalignment, wrong voltage, hold/WP pins | Verify clip alignment, power, use level-shifters, check chip detect strings with flashrom |

**Table 3: Concrete extraction failure modes, symptoms, and mitigations.**

- Record the exact JTAG pin mapping and voltage settings used for a given board — small differences (1.8V vs 3.3V) are common causes of read errors.

## A.3 Scalable hardware architectures for firmware acquisition and security

Modern approaches to hardware-assisted security show that the functions performed by FirmEx (timely anomaly detection, integrity checks, and controlled firmware acquisition) can be embedded as dedicated hardware modules or instantiated within an FPGA/SoC fabric. Below we briefly survey three complementary architectural paths and explain how each motivates a possible hardware realization of the FirmEx firmware-acquisition node.

*Architectural taxonomy.* We consider three broad approaches:

(1) **FPGA-based, configurable enclaves / TEEs.** Create isolated execution environments in the programmable logic (soft cores, BRAM-backed memory, isolated peripherals) and use hardware attestation to establish trust (e.g., BYOTee, TEEOD).
(2) **Hardware monitors / on-chip security units.** Attach programmable monitors to the CPU pipeline or bus (PHMon-style) to observe events (control-flow commits, memory accesses) and trigger hardware actions (interrupts, logging, enforcement).
(3) **Dedicated silicon RoT / secure-root modules.** Use a small verified root-of-trust chip (OpenTitan-like) that provides secure storage, measured boot, and crypto primitives; this chip can host integrity checks or mediate firmware provisioning.

*Representative works and why they matter.*

- **BYOTee** [37] demonstrates how commodity SoC-FPGA platforms can dynamically instantiate multiple, attested enclaves each with a minimal hardware TCB (softcore CPU, BRAM, isolated peripherals). BYOTee shows a practical path to move

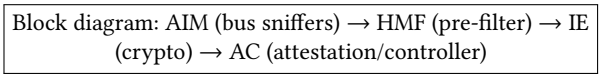


**Figure 6: Block diagram for a hardware FirmEx node (conceptual).**

security-sensitive code off the main OS and into bitstream-defined islands — a natural fit for embedding a FirmEx acquisition/verify module as an enclave that cannot be tampered with by the REE.

- **PHMon** [38] presents a programmable hardware monitor attached to a RISC-V core (prototype on FPGA) and demonstrates low overhead runtime enforcement (e.g., shadow stacks, hardware-accelerated fuzzing). PHMon quantifies area and power overheads for monitoring units and shows such monitors can perform complex security tasks with minimal runtime penalty — an attractive model for moving anomaly detection and integrity checks closer to the processor pipeline.
- **ShEF / TEEOD / related FPGA TEEs** [39, 40] explore TEEs and shielded enclaves for cloud FPGAs, presenting secure-boot, remote attestation, and shielding components for accelerators. These works illustrate real engineering trade-offs when integrating secure enclaves in multi-tenant or constrained FPGA environments (e.g., area vs. bandwidth vs. attestation complexity).
- **OpenTitan and open RoT projects** highlight the industry trend toward small, auditable silicon roots-of-trust. OpenTitan-style RoTs provide building blocks (authenticated boot, eFUSE/OTP secrets, crypto accelerators) that a FirmEx ASIC could reuse rather than implementing from scratch.

*Adapting FirmEx to hardware: a conceptual design.* Below is a concise, component-level view of how FirmEx functionality could map to a minimal hardware/SoC integration:

- **Acquisition interface module (AIM):** a small hardware IP that sits on the SoC fabric (or FPGA PL) and implements low-level bus sniffers ($I^2C$/SPI/UART/JTAG probes), basic de-serialization logic, and a small FIFO to collect captured bytes.
- **Hardware monitor / pre-filter (HMF):** programmable, rule-driven monitor (PHMon-like) that computes lightweight features (timing deltas, packet sizes, z-score windows) in hardware and raises events/interrupts on anomalies. This drastically reduces DMA/CPU traffic for trivial benign data.
- **Integrity engine (IE):** crypto unit (SHA256, MAC) that computes and verifies firmware digests in hardware; it may access secure keys stored in an on-chip RoT or eFUSE block.
- **Attestation & control (AC):** small management MCU or enclave microkernel that performs remote attestation, policy decisions, and coordinates safe dump procedures (e.g., wake the main CPU only for approved operations).

Figure 6 (suggested) can illustrate the AIM, HMF, IE and AC blocks, showing interfaces to external buses, eFUSE / RoT, and the host CPU.

*Why this is credible for FirmEx (summary).* Prototypes (BYOTee, TEEOD) show that enclave-style isolation on commodity SoC-FPGAs is practical; PHMon demonstrates hardware monitoring at low cost; OpenTitan shows how small RoT designs can provide the secure primitives needed by an integrated design. Taken together, these projects make a strong architectural case that a FirmEx firmware-acquisition and verification node can be instantiated as either an FPGA-based enclave or as a scaled-down, single-chip secure module for future work.